Guide de Déploiement Weekook

Vue d'ensemble

Weekook utilise **3 environnements** avec un déploiement automatique via GitHub Actions. Chaque environnement est lié à une **branche Git** et se déploie automatiquement à chaque `push`.

```
 LOCAL (dev)          DÉVELOPPEMENT           VALIDATION             PRODUCTION
 ─────────────        ─────────────────       ──────────             ──────────
 localhost:5173  →    dev.weekook.com    →    val.weekook.com   →    weekook.com
 localhost:3001       branche: develop        branche: staging       branche: main
                      auto-deploy ✓           auto-deploy ✓          auto-deploy +
approbation ✓
```

---

1. Les environnements

1.1 Local (développement)

Élément	Valeur
Frontend	http://localhost:5173
Backend API	http://localhost:3001
Base de données	weekook_DEV (91.99.128.31)
Lancement	`npm run dev` depuis la racine

Le frontend Vite proxifie automatiquement `/api` et `/uploads` vers le backend local (port 3001).

1.2 DEV (dev.weekook.com)

Élément	Valeur
URL	https://dev.weekook.com
Branche Git	`develop`
Serveur	91.99.128.31 (Hetzner)
Répertoire	`/var/www/weekook-dev/`
Base de données	`weekook_DEV`
Port backend	3001
Process PM2	`weekook-dev`
Workflow	`.github/workflows/deploy-dev.yml`

1.3 VAL (val.weekook.com)

Élément	Valeur

URL	https://val.weekook.com
Branche Git	`staging`
Serveur	91.99.128.31 (Hetzner)
Répertoire	`/var/www/weekook-val/`
Base de données	`weekook_VAL`
Port backend	3002
Process PM2	`weekook-val`
Workflow	`.github/workflows/deploy-val.yml`

1.4 PROD (weekook.com)

Élément	Valeur
URL	https://weekook.com
Branche Git	`main`
Serveur	91.99.128.31 (Hetzner)
Répertoire	`/var/www/weekook-prod/`
Base de données	`weekook_PRD`
Port backend	3003
Process PM2	`weekook-prod`
Workflow	`.github/workflows/deploy-prod.yml`
Particularité	**Approbation manuelle requise** via GitHub Environment `production`

---

2. Le workflow Git

2.1 Branches

```
develop    ← développement quotidien, auto-deploy sur dev.weekook.com
   ↓ merge
staging    ← validation client/métier, auto-deploy sur val.weekook.com
   ↓ merge
main       ← production, auto-deploy + approbation manuelle sur weekook.com
```

2.2 Flux de travail quotidien

Développer une fonctionnalité :

```
# 1. Travailler sur develop (ou une feature branch)
git checkout develop
```

```
# 2. Coder, tester en local
npm run dev

# 3. Committer et pousser
git add .
git commit -m "feat: description de la fonctionnalité"
git push origin develop
# → GitHub Actions déploie automatiquement sur dev.weekook.com (≈30s)
```

Promouvoir en validation :

```
# 1. Se placer sur staging
git checkout staging

# 2. Merger develop dans staging
git merge develop

# 3. Pousser
git push origin staging
# → GitHub Actions déploie automatiquement sur val.weekook.com (≈30s)
```

Promouvoir en production :

```
git checkout main
git merge staging
git push origin main
# → GitHub Actions crée un run en attente d'approbation
# → Aller sur GitHub → Actions → Cliquer "Review deployments" → Approuver
# → Le déploiement s'exécute sur weekook.com
```

2.3 Résumé en une commande

```
# Déployer sur DEV :
git push origin develop

# Déployer sur VAL :
git checkout staging && git merge develop && git push origin staging

# Déployer sur PROD (nécessite approbation dans GitHub Actions) :
git checkout main && git merge staging && git push origin main
```

---

3. GitHub Actions — Comment ça marche

3.1 Principe

Chaque environnement a un fichier workflow dans `.github/workflows/` :

Fichier	Déclenché par	Déploie sur
`deploy-dev.yml`	Push sur `develop`	dev.weekook.com
`deploy-val.yml`	Push sur `staging`	val.weekook.com
`deploy-prod.yml`	Push sur `main` + approbation	weekook.com

3.2 Ce que fait un déploiement

Quand on pousse sur une branche, GitHub Actions exécute ces étapes **sur le serveur via SSH** :

```
1. git pull origin <branche>         ← Récupère le dernier code
2. npm install --workspaces          ← Installe/met à jour les dépendances
3. npx prisma generate               ← Régénère le client Prisma
4. npx prisma db push --accept-data-loss ← Applique les changements de schéma
5. npm run build                      ← Build client (Vite) + server (TypeScript)
6. pm2 reload <process>              ← Redémarre le backend sans downtime
```

Tâches automatiques au démarrage du backend (à chaque `pm2 reload`, sur tous les environnements) :

Tâche	Rôle
`seedConfig()`	Crée les valeurs de configuration par défaut manquantes (n'écrase jamais l'existant)
`normalizeKookerSpecialties()`	Convertit en listes JSON les spécialités kooker stockées en texte
`backfillKookerCoordinates()`	En arrière-plan : géolocalise les kookers qui ont une ville mais pas de coordonnées

Ces tâches sont idempotentes : elles ne font rien si les données sont déjà à jour.

Accès réseau sortant requis : le serveur appelle `https://geo.api.gouv.fr` et `https://data.geopf.fr` (localisation des adresses des kookers et de l'adresse de recherche des clients, autocomplétion). Si ces API sont injoignables, l'autocomplétion et la recherche par adresse échouent (message « adresse introuvable »), mais le reste du site fonctionne. Délai de 8 s avec une nouvelle tentative.

3.3 Secrets GitHub

Les workflows utilisent des secrets configurés dans **GitHub → Settings → Secrets** :

Secret	Description	Valeur
`SSH_HOST`	IP du serveur	91.99.128.31
`SSH_USER`	Utilisateur SSH	admin
`SSH_PRIVATE_KEY`	Clé privée SSH (RSA/Ed25519)	Contenu de la clé privée

Ces secrets sont **partagés** entre tous les workflows (même serveur).

3.4 Vérifier un déploiement

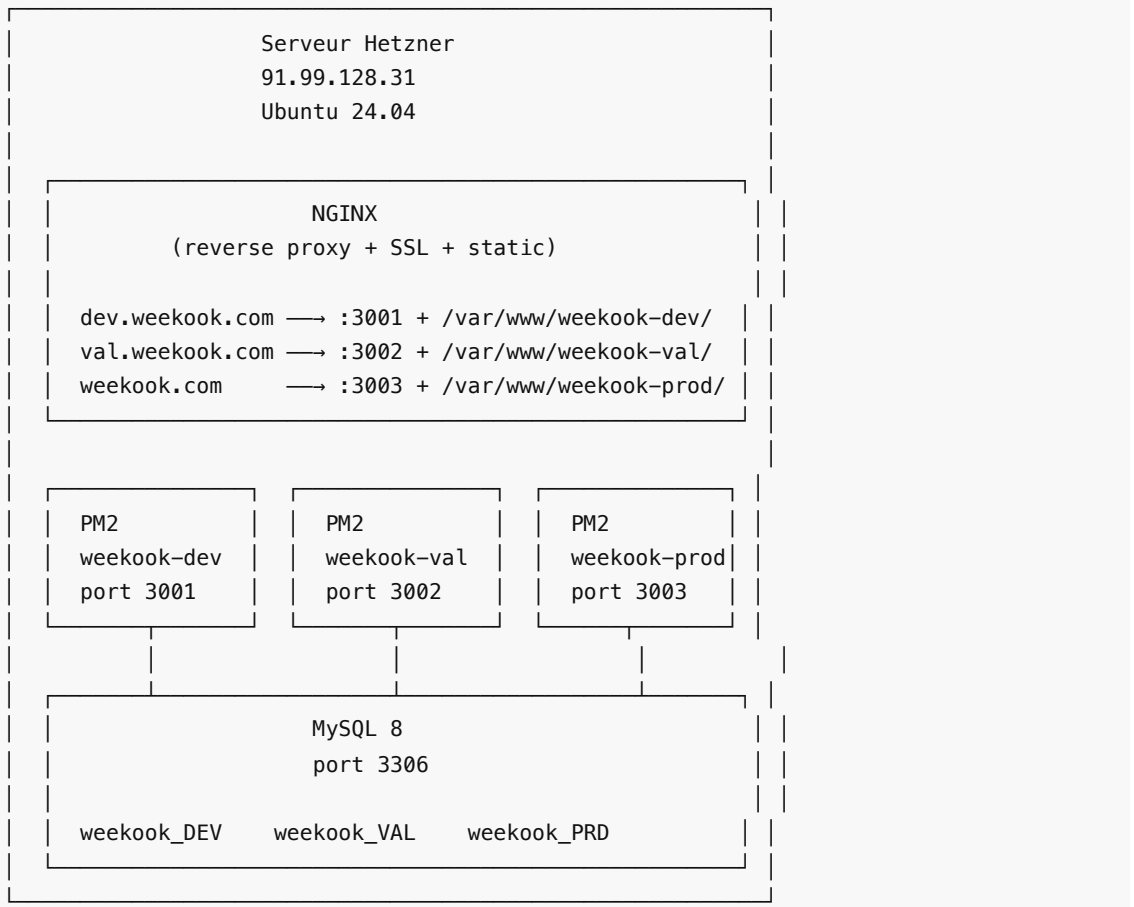
1. Aller sur **GitHub → Actions** : https://github.com/guillaumeroca/weekook/actions
2. Cliquer sur le dernier run pour voir les logs
3. Icône verte ✓ = succès, rouge ✗ = échec

En cas d'échec, cliquer sur le job pour voir l'erreur dans les logs.

---

4. Architecture serveur

4.1 Vue d'ensemble



4.2 Nginx

Chaque environnement a sa config Nginx :

```
/etc/nginx/sites-available/weekook-dev    → dev.weekook.com
/etc/nginx/sites-available/weekook-val    → val.weekook.com
/etc/nginx/sites-available/weekook.com    → weekook.com (+ www redirect)
```

Nginx gère :

- **SSL** (certificats Let's Encrypt via Certbot, renouvellement auto)
- **Static files** : sert le frontend buildé (`client/dist/`)
- **Reverse proxy** : redirige `/api/` et `/uploads/` vers le backend Node.js

- **SPA fallback** : renvoie `index.html` pour toutes les routes frontend

4.3 PM2

PM2 gère les processus Node.js backend :

```
# Voir l'état de tous les processus
pm2 status

# Logs d'un processus
pm2 logs weekook-dev
pm2 logs weekook-val
pm2 logs weekook-prod

# Redémarrer manuellement
pm2 reload weekook-dev
pm2 reload weekook-val
pm2 reload weekook-prod

# Sauvegarder la config PM2 (persist au reboot)
pm2 save
```

4.4 Fichiers .env serveur

Chaque environnement a son propre `.env` (jamais committé) :

```
/var/www/weekook-dev/.env    → config DEV
/var/www/weekook-val/.env    → config VAL
/var/www/weekook-prod/.env   → config PROD
```

Variables clés qui changent entre environnements :

Variable	DEV	VAL	
DATABASE_URL	mysql://...@localhost/weekook_DEV	mysql://...@localhost/weekook_VAL	mysc
PORT	3001	3002	3003
CORS_ORIGIN	https://dev.weekook.com	https://val.weekook.com	https
COOKIE_DOMAIN	dev.weekook.com	val.weekook.com	week
APP_URL	https://dev.weekook.com	https://val.weekook.com	https
JWT_SECRET	(secret unique DEV)	(secret unique VAL)	(secr

---

5. Opérations courantes

5.1 Déployer une correction urgente

```
# Fix sur develop
git checkout develop
```

```
# ... faire le fix ...
git add . && git commit -m "fix: description" && git push origin develop
# → DEV mis à jour

# Propager immédiatement en VAL
git checkout staging && git merge develop && git push origin staging
# → VAL mis à jour

# Propager en PROD (si validé en VAL)
git checkout main && git merge staging && git push origin main
# → Approuver dans GitHub Actions → PROD mis à jour
```

5.2 Vérifier l'état du serveur

```
ssh admin@91.99.128.31

# État des processus
pm2 status

# Logs en temps réel
pm2 logs weekook-dev --lines 50
pm2 logs weekook-val --lines 50

# Vérifier Nginx
sudo nginx -t
sudo systemctl status nginx

# Vérifier les certificats SSL
sudo certbot certificates
```

5.3 Rollback (retour arrière)

```
# Sur le serveur, revenir au commit précédent
ssh admin@91.99.128.31
cd /var/www/weekook-dev   # ou weekook-val
git log --oneline -5      # voir les derniers commits
git checkout <commit-hash>
npm run build
pm2 reload weekook-dev
```

Puis corriger le code en local et re-pousser.

5.4 Modifier le schéma de base de données

```
# 1. Modifier prisma/schema.prisma en local
# 2. Tester en local
npx prisma db push
```

```
# 3. Pousser sur develop → le workflow exécute automatiquement prisma db push
git add . && git commit -m "schema: description" && git push origin develop
```

Attention : `prisma db push --accept-data-loss` dans les workflows peut supprimer des colonnes. Pour la production, utiliser `prisma migrate` à la place.

5.5 Ajouter une variable d'environnement

1. Se connecter en SSH : `ssh admin@91.99.128.31`
2. Éditer le `.env` de l'environnement : `nano /var/www/weekook-dev/.env`
3. Redémarrer le processus : `pm2 reload weekook-dev`

5.6 Consulter les logs d'un déploiement GitHub Actions

1. Aller sur https://github.com/guillaumeroca/weekook/actions
2. Cliquer sur le workflow run concerné
3. Cliquer sur le job `deploy`
4. Lire les logs étape par étape

---

6. Dépannage

Le déploiement GitHub Actions échoue

Symptôme	Cause probable	Solution
`Permission denied (publickey)`	Clé SSH invalide	Vérifier le secret `SSH_PRIVATE_KEY` dans GitHub
`npm install` échoue	Dépendance cassée	Se connecter en SSH et lancer manuellement
`prisma db push` échoue	Conflit de schéma	Se connecter en SSH et résoudre manuellement
`pm2 reload` échoue	Process inexistant	Se connecter en SSH et `pm2 start`
`git pull` échoue	Conflits locaux	Se connecter en SSH et `git reset --hard origin/<branche>`

Le site ne répond pas

```
ssh admin@91.99.128.31

# 1. Vérifier PM2
pm2 status
# Si le process est "errored" :
pm2 logs weekook-dev --lines 100

# 2. Vérifier Nginx
sudo systemctl status nginx
sudo nginx -t

# 3. Vérifier le port
```

```
curl http://localhost:3001/api/v1/auth/me   # DEV
curl http://localhost:3002/api/v1/auth/me   # VAL
curl http://localhost:3003/api/v1/auth/me   # PROD
```

Problème de certificat SSL

```
# Renouveler manuellement
sudo certbot renew

# Vérifier l'expiration
sudo certbot certificates
```

---

7. Checklist — Ajouter un nouvel environnement

Pour référence, voici les étapes pour créer un nouvel environnement (ex: PROD) :

1. **DNS** : Ajouter un enregistrement A `weekook.com → 91.99.128.31`
2. **MySQL** : Créer la base `weekook_PRD` et son utilisateur
3. **Clone** : `git clone` dans `/var/www/weekook-prod/`, checkout `main`
4. **.env** : Créer le fichier avec les variables de l'environnement
5. **Build** : `npm install && npx prisma generate && npx prisma db push && npm run build`
6. **PM2** : `pm2 start server/dist/app.js --name weekook-prod && pm2 save`
7. **Nginx** : Créer la config dans `/etc/nginx/sites-available/`, activer, reload
8. **SSL** : `sudo certbot --nginx -d weekook.com`
9. **GitHub Actions** : Créer `.github/workflows/deploy-prod.yml` sur la branche `main`
10. **Tester** : Ouvrir le site et vérifier